

RAG sistem za pretragu naučnih radova sa fine-tuningom embedding modela

Nemanja Ljubičić, E9 8/2025, nemanjaljubice2@gmail.com

I. UVOD

RAG (Retrieval-Augmented Generation) je tehnika koja je osmišljena tako da obezbedi velikim jezičkim modelima pristup spoljnoj bazi podataka i njenu pretragu.

Primećeno je da veliki jezički modeli, prilikom davanja odgovora, često daju netačne i izmišljene informacije. Takođe, njihovo znanje je ograničeno na skup podataka koji se koristio prilikom treninga.

Ideja RAG-a je da obezbedi pristup novim, specifičnim ili neviđenim podacima prilikom treninga i da se odgovor korisniku formuliše na osnovu znanja dobijenog iz tih podataka, postupkom *retrieval*-a. Ovo je posebno korisno, jer se navodi i direktan dokument odakle je informacija pribavljena.

Jedna od glavnih komponenti RAG-a je pomenuti retrieval, pa se pristup primenjen u ovom projektu oslanja na postojeća saznanja o retrievalu na skupu koji će biti korišćen, jer je problem verifikacije naučnih tvrdnji na SciFact skupu deo šireg BEIR benchmarka [1]. BEIR pokazuje da klasičan metod BM25 predstavlja robustan baseline, dok gusti retriveri često podbacuju u odnosu na njega, posebno kada postoji veliki domenski pomak u odnosu na podatke na kojima su trenirani. Ista studija pokazuje da domenska adaptacija poboljšava rezultate baš na naučnim publikacijama. Ovaj nalaz motiviše pristup u ovom projektu, gde se umesto oslanjanja na pretreniran embedding model, taj isti model fino podešava na domenske parove, čime se gusti retrieval prilagođava specifičnoj terminologiji korpusa.

II. POSTAVKA PROBLEMA

U ovom projektu, dostupni su parovi upit-rad. Cilj je, na osnovu datog upita, pronaći rad koji se odnosi na taj upit. Ovaj deo se naziva *retrieval*. Potom izvršiti potvrdu ili negiranje tvrdnje i na kraju, ispisati odgovor korisniku.

Za evaluaciju *retrieval* sistema biće korišćene sledeće metrike:

1. NDCG@10 (*Normalized Discounted Cumulative Gain*): meri kvalitet rangiranja prvih 10 rezultata. Relevantni dokumenti rangirani više nose veću vrednost, a oni niže doprinose manje. Vrednost se normalizuje idealnim rasporedom, pa se kreće između 0 i 1.
2. Recall@10: Meri udeo relevantnih dokumenata pronađenih u prvih 10 rezultata, u odnosu na ukupan broj relevantnih za taj upit. Pokazuje koliko je sistem široko zahvatio.

3. MRR@10 (*Mean Reciprocal Rank*): meri koliko visoko je prvi relevantan dokument rangiran. Za svaki upit uzima se recipročna vrednost pozicije prvog relevantnog rezultata (1 za prvo mesto, 1/3 za treće itd.), a MRR je prosek tih vrednosti preko svih upita.

Ove tri metrike se koriste zajedno jer mere različite aspekte: NDCG ocenjuje kvalitet celog redosleda, Recall koliko je od svih relevantnih dokumenata uopšte uhvaćeno, a MRR koliko brzo dolazi prvi relevantan rezultat.

III. BAZA PODATAKA

Baza podataka koja je korišćena je BEIR SciFact [2]. To je stručno anotiran korpus namenjen verifikaciji naučnih tvrdnji.

Konkretno, baza se sastoji od tri komponente:

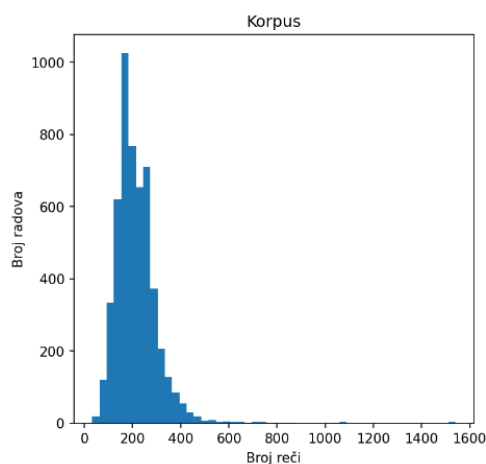
Korpus koji sadrži naslove i tekstove dokumenata (ukupno 5.183 rada), skup upita u vidu naučnih tvrdnji i skup oznaka relevantnosti koji definiše koji dokumenti podržavaju ili opovrgavaju koje tvrdnje.

Dakle, radi se o bazi u kojoj svaki dokument predstavlja kratak naučni apstrakt, a svaki upit predstavlja naučnu tvrdnju koju treba proveriti pronalaženjem relevantnih dokaza u korpusu.

Postoji ukupno 1109 upita, 919 trening parova upit-rad.

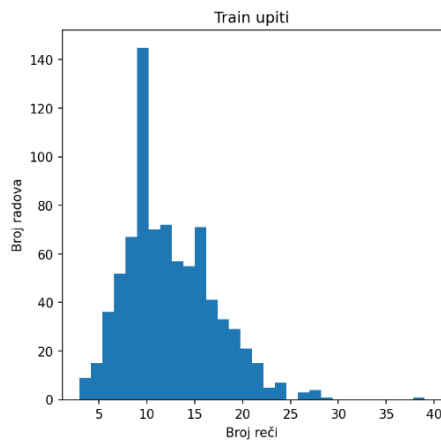
Potrebno je proveriti i dužinu, tj. broj reči koje imaju upiti i radovi.

Slika III.1. prikazuje histogram broja reči u radovima. Minimalan broj reči je 33, maksimalan 1541, a prosečan 214.6.



Slika III.1 Histogram dužine radova

Slika III.2. prikazuje histogram broja reči upita. Minimalan broj reči je 3, maksimalan 39, a prosečan 12.3.



Slika III.2 Histogram dužine upita

IV. PODELA SKUPA

Podela skupa rađena je na nivou upita, a ne na nivou pojedinačnih upit-dokument parova, kako bi se sprečilo curenje informacija između trening i validacionog skupa (*data leak*).

Razlog je što jedan upit (tvrdnja) može biti povezan sa više relevantnih dokumenata, pa bi deljenje po parovima dovelo do situacije da se isti upit nađe istovremeno i u trening i u validacionom skupu, samo sa različitim pridruženim dokumentima. U tom slučaju model bi tokom treniranja već video tvrdnje na kojima se kasnije validira, čime bi rezultati validacije bili optimistični. Test skup bi svakako ostao netaknut, ali validacija više ne meri sposobnost modela da se nosi sa neviđenim tvrdnjama, već delom i ono što je već naučio.

Iz tog razloga deljena je lista identifikatora upita, nakon čega su iz dobijenih podskupova rekonstruisani odgovarajući upiti i relevantni dokumenti. Na taj način svaki upit u celosti, zajedno sa svim svojim parovima, pripada isključivo jednom skupu.

Ispravnost ovakve podele potvrđena je proverom preseka između skupova, koji je u sva tri slučaja (trening presek validacija, trening presek test, validacija presek test) jednak nuli, što znači da nijedan upit ne curi iz jednog skupa u drugi.

Originalni trening skup sadrži 919 parova upit-dokument, pri čemu je za potrebe fine-tuninga podeljen na dva dela: 827 parova za fine-tuning i 92 para za validaciju. Test skup sadrži 339 parova upit-dokument.

V. VEKTOR EMBEDDING

U domenu obrade prirodnog jezika (NLP), računarski sistemi nisu u mogućnosti da direktno obrade tekstualne podatke u njihovom izvornom obliku.

Iz tog razloga, neophodno je izvršiti transformaciju teksta u numerički format koji je pogodan za matematičke operacije.

Vektorski embedding je postupak mapiranja tekstualnog ulaza (reč, rečenica ili ceo tekst) u višedimenzionalni prostor realnih brojeva.

Kao rezultat ovog postupka dobija se vektor fiksne dimenzionalnosti koji opisuje semantičke karakteristike

ulaznog teksta.

Formalno, za ulaznu rečenicu s , embedding model f vrši sledeće preslikavanje:

$$f(s) \rightarrow \mathbb{R}^n$$

Gde n označava dimenzionalnost vektorskog prostora.

Važna osobina ovakvog preslikavanja je da se slični iskazi preslikavaju u vektore koji su međusobno geometrijski slični. Ova bliskost između dva vektora izražava preko kosinusne sličnosti:

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \cdot \|B\|}$$

Vrednost kosinusne sličnosti se kreće u intervalu $[-1, 1]$.

Za posmatrane vektore A i B , vrednost bliska 1 ukazuje na visoku semantičku sličnost, vrednost oko 0 na semantičku nepovezanost iskaza, a vrednost bliska -1 na semantičku suprotnost.

A. Model all-MiniLM-L6-v2

Model all-MiniLM-L6-v2 [3] je vektor embedding model razvijen u okviru *Sentence Transformers* frejmworka i zasnovan je na Majkrosoftovoj *MiniLM* arhitekturi. Njegova arhitektura podrazumeva tri faze procesiranja ulaznog teksta:

1. Tokenizacija: ulazni tekst se prevodi u niz podreči (tokena), pri čemu maksimalna dužina ulaznog niza iznosi 256 tokena. Sekvence koje premašuju ovaj limit bivaju skraćene. Ovaj model koristi *WordPiece* tokenizaciju (isti algoritam koji koristi BERT). Princip rada zasniva se na ideji da česte reči dobiju sopstveni token, dok se retke reči razlažu na česte podreči. *Wordpiece* ima fiksni rečnik od 30.522 tokena, a svaka moguća reč koja se pojavi na ulazu, može se izraziti kombinacijom tih fragmenata. Na početku svake sekvence je klasifikacioni token (CLS), a na kraju separacioni token (SEP).
2. Kontekstualno enkodovanje: Nakon tokenizacije, prethodno dobijena sekvenca prolazi kroz 6 transformer slojeva. U svakom sloju, svaki token računa težine prema svim ostalima i ažurira svoju reprezentaciju kao ponderisanu kombinaciju. Ovo je mehanizam pažnje. Niži slojevi uočavaju jednostavnije odnose, dok oni viši slojevi hvataju finije zavisnosti. Po prolasku kroz svih 6 slojeva, svaki token poseduje vektorsku reprezentaciju koja u sebi nosi ne samo sopstveno značenje (koje je imao na početku), već i ulogu koju ima u kontekstu rečenice. Ovakvim pristupom token nije jedan fiksni vektor, već može imati različite reprezentacije u odnosu na rečenicu u kojoj je upotrebljen. To je glavna prednost u odnosu na prethodne modele (kao što je npr. *Word2Vec*).
3. *Mean pooling*: Nakon što su svi tokeni prošli kroz 6 transformer slojeva, na raspolaganju je skup vektorskih reprezentacija, odnosno po jedan vektor dimenzionalnosti 384 za svaki token u sekvenci. Problem koji se sada javlja jeste sledeći: različite rečenice imaju različit broj tokena, a za dalju

upotrebu neophodan je jedan vektor fiksne dimenzionalnosti koji predstavlja celu rečenicu. Budući da se rečenice procesiraju u grupama a različite rečenice imaju različitu dužinu, kraće rečenice se dopunjuju *padding* tokenima kako bi sve sekvence u grupi imale jednaku dužinu. Ti tokeni ne nose nikakvu semantičku informaciju i ne smeju učestvovati u izračunavanju proseka. Iz tog razloga koristi se binarna maska koja modelu eksplicitno označava koji tokeni su stvarni sadržaj, a koji su *padding*. *Mean pooling* uzima prvu dimenziju svih vektora i usrednjava je, zatim drugu dimenziju i usrednjava je, i tako redom za svih 384 dimenzija. Rezultat je jedan vektor od 384 broja.

4. L2 normalizacija: Dobijeni vektor od 384 dimenzija se na kraju normalizuje na jediničnu dužinu, tako što se svaka komponenta podeli normom (dužinom) vektora. Posle ovog koraka svi vektori leže na jediničnoj sferi, tj. imaju dužinu 1. To olakšava poređenje, jer kada vektori imaju jediničnu dužinu, kosinusna sličnost između dva vektora svodi se na njihov skalarni proizvod. To pojednostavljuje i ubrzava pretragu, jer se umesto pune formule za kosinusnu sličnost može računati samo skalarni proizvod.

B. Rolling window

Pre nego što sistem može da odgovara na upite, neophodno je izvršiti obradu celokupnog korpusa od 5.183 apstrakta, metodom opisanom u prethodnom odeljku.

Problem koji ovde postoji je ograničenje modela na 256 tokena na ulazu, a ranija analiza je pokazala da je prosečan broj reči 215. To je približno 280 tokena, s tim da je s obzirom na prirodu dataseta taj broj vrlo verovatno i veći. To znači da će svi tokeni koji premaše 256 biti odsečeni, odnosno gubi se deo teksta. Zbog toga model ne raspolaže celim tekstom, a moguće je da se bitna informacija nalazi baš na kraju.

Zato se koristi metoda *rolling window*. Definiše se veličina prozora (broj tokena u svakom prozoru, *window*), kao i korak pomeranja prozora (*stride*).

Ovi parametri određeni su empirijski (*window* = 190, *stride* = 100), na osnovu karakteristika korpusa i ograničenja *embedding* modela. Vrednost od 190 tokena iskorišćava većinski deo kapaciteta modela, ali zadržava rezervu u slučaju varijacija u tokenizaciji. Korak od 100 tokena znači da se susedni prozori preklapaju za 90 tokena. To je namerno uvedeno jer garantuje da informacija na granici između dva prozora bude sadržana u celosti u bar jednom od njih.

Premali prozor bi učinio da nijedan vektor ne predstavlja kompletnu informaciju, dok prevelik prozor vraća problem sečenja i kroz *mean pooling* razvodnjava specifičnu informaciju u proseku celog teksta. Slično, premali korak stvara veliki broj identičnih prozora, a prevelik korak ostavlja granične celine nepokrivene.

Korišćen je tokenizer specifičan za *all-MiniLM-L6-v2*, što garantuje konzistentnost između prozorovanja i kasnijeg embedovanja.

Postupak obezbeđuje da, za radove veće od 256 tokena,

postoji više vektora koji ga opisuju.

Početni broj dokumenata je pomenutih 5.183, a vektorska baza nakon ovog postupka sadrži 15.261 vektor. Svaki od delova teksta je mapiran na originalni rad od kog je potekao.

C. Postupak pretrage dataseta vektor embedding modelom

Korisnik postavlja naučnu tvrdnju čiju tačnost želi proveriti.

Upit prolazi kroz identičan proces kao i dokumenti tokom indeksiranja i takođe se pretvara u vektor dimenzionalnosti 384. Isti model koristi za enkodovanje i dokumenata i upita. Na taj način vektori upita i dokumenata postoje u istom vektorskom prostoru i mogu se porediti.

Bliskost se meri pomenutom kosinusnom sličnošću između vektora upita i vektora u bazi. Da bi se ovo poređenje izvelo efikasno, koristi se FAISS indeks, biblioteka za brzu pretragu nad velikim skupom vektora. *Rolling window* je povećao broj vektora (oko 3 puta), ali FAISS baza uspešno obrađuje i prostore određene sa milionima vektora.

Zadatak FAISS-a je da pronađe delove tekstova (prozore) čiji su vektori geometrijski najbliži vektoru upita, što odgovara semantičkoj srodnosti sadržaja. Kako je svaki prozor teksta mapiran na originalni rad, onda se korisniku može vratiti deo rada (tj. prozor) ili ceo odgovarajući naučni rad.

VI. BASELINE MODEL

Kao polazna tačka za treniranje *all-MiniLM-L6-v2* korišćen je pre-trenirani model *nreimers/MiniLM-L6-H384-uncased*, koji je već naučio osnove jezika kroz velikim tekstualnim korpusima. Za *fine-tuning* ovog modela korišćeni su naučni tekstovi, forumske diskusije, pretraživački upiti, fotografski opisi, pitanja i odgovori, novinarski tekstovi.

Svrha ovako šarenog skupa podataka je da se osigura opšta semantička kompetencija modela, odnosno da ima sposobnost da razume sličnost između rečenica bez obzira na domen.

I pored impresivnog opšteg treninga, *all-MiniLM-L6-v2* poseduje ograničenje kada se primenjuje na specifičnim domenima poput biomedicinske nauke, prava, finansija ili tehničke dokumentacije. Ovakvi usko specijalizovani domeni poseduju sopstvenu terminologiju i skraćenice koje se nisu našle tokom treninga modela. Zato je koristan *fine-tuning* za užu oblast za koju će se model koristiti. Time se postiže

1. Rekalibracija vektorskog prostora: matrice težina u svim slojevima modela prilagođavaju se tako da terminologija domena dobija korigovanu poziciju u 384-dimenzionalnom prostoru.
2. Prilagođavanje distribuciji upita: model uči odnos između formata upita i formata dokumenata koji postoji u ciljnom skupu podataka. Za SciFact, to znači učenje odnosa između kratkih naučnih tvrdnji i dugih apstrakata.

Posledično, to dovodi do toga da se poboljšava RAG: relevantni dokumenti dobijaju više kosinusne sličnosti sa

upitom, što kao posledicu ima tačnije rangiranje u fazi pretraživanja vektorske baze.

VII. FINE TUNE VEKTOR EMBEDDING MODELA

Za pretragu prostora hiperparametara korišćena je biblioteka Optuna sa TPE samplerom (*Tree-structured Parzen Estimator*). To je algoritam zasnovan na Bajesovskoj optimizaciji i radi u nekoliko koraka:

1. Prvih 10 *trial*-ova (default u Optuni) se odabira nasumično.
2. Za svaki naredni *trial* se svi dotad završeni dele na dobre (gornji kvantil, default 10%) i loše (ostatak)
3. Za svaku grupu se gradi probablistički model raspodele parametara.
4. Sledeći *trial* bira parametre koji su verovatni u dobroj grupi i neverovatni u lošoj.
5. Sa svakim novim *trial*-om model postaje precizniji tako što rane *trial*-e istražuje široko, kasnije eksploatiše dobre zone.

Optuni se zadaje prostor pretrage, definisan sa željenim hiperparametrima i opsezima vrednosti koje parametri mogu da imaju.

Za potrebe projekta korišćena je *Sentence Transformers* biblioteka i njihov model `_trial.fit()`, koja podrazumevano koristi *AdamW* optimizator.

Za početni opseg *learning rate*-a, autorova preporuka je $2e-5$, pa je opseg ($1e-6$, $1e-4$), sa logaritmskim uzorkovanjem.

Opseg za batch size je (8, 256), broj epoha (1, 20).

Autorovih 500 koraka od 100.000 ukupnih predstavlja 0.5%. Zbog manjeg skupa podataka *warmup* je ovde od 0 do 10%, (0.0,0.1).

Standardna podrazumevana vrednost *AdamW weight decay*-a je 0.01. Istražuje se opseg od 0 do 0.1.

Originalni model je fino podešavan primenom kontrastivnog cilja. Formalno, izračunava se kosinusna sličnost za svaki mogući par rečenica u batch-u, a zatim se primenjuje cross-entropy loss poređenjem sa stvarnim parovima.

MultipleNegativesRankingLoss [4], kao jedna od najkorišćenijih *loss* funkcija, implementira ovaj princip za svaki pozitivan par u batch-u, svi ostali primeri tretiraju se kao negativni, što odgovara originalnoj proceduri.

Za treniranje se koriste isključivo parovi sa oznakom SUPPORT, jer jedino oni predstavljaju stvarnu semantičku podudarnost upita i dokumenta potrebnu za kontrastivno učenje. Parovi sa oznakom CONTRADICT i NOINFO se izostavljaju, jer ne predstavljaju validne pozitivne primere za ovaj tip treninga.

InformationRetrievalEvaluator je standardni evaluator *Sentence Transformers* biblioteke namenjen ovakvim zadacima.

Autori BEIR benchmarka [1] navode da je važno koristiti jednu evaluacionu metriku koja se može uporedivo izračunati na svim zadacima. Metrike poput *Precision* i *Recall* koje su nesvesne ranga nisu pogodne. Binarne metrike svesne ranga poput MRR i MAP ne uspevaju da evaluiraju zadatke sa gradiranim relevantnim ocenama.

Ranije pomenuti NDCG je pogodan za zadatke koji uključuju i binarne i gradirane relevantne ocene. Svi

eksperimenti u BEIR radu koriste $nDCG@10$.

Pokrenuto je 100 *trials*-a pretrage, sa opisanim podešavanjima. Najboljih 5 *trials*a nakon pretrage, i njihovi hiperparametri su dati u Tabeli VII.1.

Tabela VII.1 Najboljih 5 trialsa prve pretrage HPO, sa vrednostima parametara

Trial	Ep	LR	Batch	Warmup	WD
94	14	9.97e-05	128	0.008	0.093
90	14	9.92e-05	128	0.015	0.095
95	14	9.98e-05	128	0.007	0.092
93	14	9.90e-05	128	0.008	0.092
85	14	9.90e-05	128	0.012	0.100

Primećuje se da su dva bitna parametra na granici pretrage, a to su *learning rate* i *weight decay*.

Vrednosti metrika ostvarenih na trening i na validacionom skupu su date u Tabeli VII.2.

Tabela VII.2 Najboljih 5 trialsa prve pretrage HPO, s vrednostima metrika na trening i validacionom skupu

	NDCG	Recall	MRR
Trial	Train Val	Train Val	Train Val
94	0.9501 0.700	0.9845 0.7677	0.9369 0.7058
90	0.9496 0.6999	0.9839 0.7677	0.9368 0.7060
95	0.9501 0.6998	0.9845 0.7677	0.9369 0.7059
93	0.9498 0.6997	0.9845 0.7677	0.9362 0.7058
85	0.9497 0.6997	0.9839 0.7677	0.9370 0.7054

Takođe, primećuje se i train-val jaz u metrikama. S obzirom na to da validacioni skup sadrži 92 para, ovaj jaz se najverovatnije može pripisati varijansi malog validacionog skupa, a ne *overfitting*-u modela.

Izvršena je druga pretraga sa proširenim opsegom *weight decay* (0.0,0.3) i *learning rate* od $1e-6$ do $1e-3$.

Druga pretraga je dala sledeće parametre kao najbolje (Tabela VII.3) i vrednosti na trening i validacionom skupu (Tabela VII.4)

Tabela VII.3 Najboljih 5 trialsa druge pretrage HPO, sa vrednostima parametara

Trial	Ep	LR	Batch	Warmup	WD
96	17	4.54e-04	256	0.092	0.120
85	18	4.58e-04	256	0.088	0.127
31	19	6.07e-04	256	0.092	0.173
87	18	4.62e-04	256	0.092	0.112
33	19	5.92e-04	256	0.092	0.142

Tabela VII.4 Najboljih 5 trials-a druge pretrage HPO, s vrednostima metrika na trening i validacionom skupu

	NDCG	Recall	MRR
Trial	Train Val	Train Val	Train Val
96	0.9773 0.7383	0.9916 0.7970	0.9693 0.7522
85	0.9783 0.7377	0.9909 0.7846	0.9720 0.7558
31	0.9810 0.7356	0.9910 0.7963	0.9773 0.7368
87	0.9778 0.7350	0.9911 0.7784	0.9700 0.7557
33	0.9812 0.7339	0.9917 0.7908	0.9782 0.7333

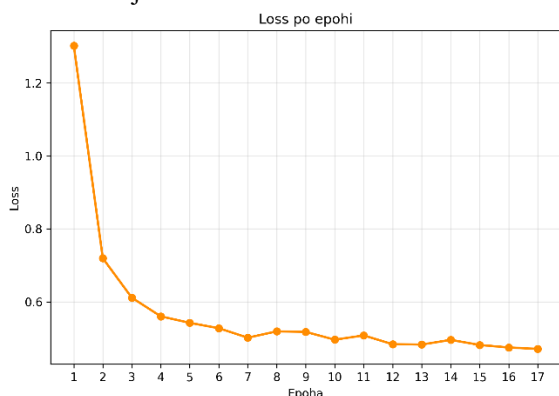
Batch size i *warmup ratio* su na ivicama opsega, ali oni pokazuju blaže i monotonije ponašanje kroz pretragu, pa njihov izbor manje utiče na finalni model i nije bio kritičan za dodatno podešavanje.

VIII. TRENIRANJE I EVALUACIJA MODELA

Treniranje modela je u skladu sa definisanom bazom preko metode *rolling window* i korišćenjem funkcije gubitka *MultipleNegativesRankingLoss*. Korišćen je prošireni trening skup, odnosno trening skupu su sada dodati i validacioni parovi.

Budući da su validacioni parovi uključeni u skup za obučavanje, prati se isključivo trening gubitak, koji služi za proveru konvergencije modela, a ne za procenu generalizacije.

Slika VIII.1 prikazuje prosečnu vrednost funkcije gubitka za svaku od 17 epoha. Gubitak naglo opada tokom prve dve epohe, nastavlja da opada postupno do otprilike sedme epohe, nakon čega se stabilizuje na vrednosti oko 0.45 uz manje oscilacije. Ovakav oblik krive ukazuje na stabilnu konvergenciju i da nema znakova nestabilnosti tokom obučavanja.



Slika VIII.1 Loss po epohi, tokom treninga

Performanse su procenjene na nezavisnom test skupu.

Porede se pretrenirani all-MiniLM-L6-v2 koji koristi bazu na kojoj prethodno nije obavljen *rolling window* (originalna baza, odnosno jedan dokument dobija jedan

vektor), pretrenirani all-MiniLM-L6-v2 sa bazom na koju je primenjen *rolling window* i *fine-tune*-ovan all-MiniLM-L6-v2, na *rolling window* bazi. Rezultati su u Tabeli VIII.1.

Tabela VIII.1 Vrednosti metrika na test skupu, za tri različita pristupa

all-MiniLM-L6-v2	NDCG	Recall	MRR
Pretrenirani + Originalna baza	0.6451	0.7833	0.6047
Pretrenirani + <i>Rolling window</i> baza	0.6718	0.8206	0.6260
<i>Fine-tuned</i> + <i>Rolling window</i> baza	0.7818	0.8840	0.7520

Tabela pokazuje napredak kroz sve tri varijante modela. Sama primena *rolling window* metode donosi blagi porast svih metrika, što potvrđuje da pozitivno utiče na *retrieval* i pre ikakvog treniranja.

Glavni doprinos dolazi od fine-tuninga, gde NDCG raste sa 0.6718 na 0.7818, Recall sa 0.8206 na 0.8840, a MRR sa 0.6260 na 0.7520.

Takođe, test metrike fine-tunovanog modela (NDCG 0.782) više su od validacionih (~0.738) iz HPO faze. Da je train-val jaz bio posledica *overfitting*-a, model bi na nezavisnom test skupu ostvario slabiji rezultat. Test rezultat je viši i to ide u prilog ranijoj pretpostavci da jaz potiče od varijanse malog validacionog skupa.

Ovo bi moglo biti rešeno korišćenjem unakrsne validacije (sa npr. 5 *fold*-ova), koja bi obezbedila stabilnije rezultate. Problem je što bi, već poprilično vremenski dugačka HPO pretraga, trajala 5 puta duže.

IX. RAG SISTEM

RAG sistem sastoji se od dve komponente: *retriever*-a, koji pronalazi relevantne dokumente iz korpusa, i generatora, koji na osnovu pronađenih dokumenata i upita generiše odgovor.

Generator je jezički model FLAN-T5-XL, koji prima prompt sa kontekstom i tvrdnjom i proizvodi odgovor. To je model iz T5 familije, *encoder-decoder* arhitekture, sa oko 3 milijarde parametara. Za razliku od čisto generativnih modela, T5 je *seq2seq* model treniran da na osnovu tekstualnog ulaza proizvede tekstualni izlaz, a FLAN varijanta je dodatno trenirana na velikom broju zadataka zadatih prirodnim jezikom. Model ima ograničenu dužinu ulaza, koja je preporučena na 512 tokena. To opravdava i format prompta koji će biti korišćen.

Sam proces se odvija u nekoliko koraka:

1. Dovlačenje dokumenata. Za zadatak tvrdnju prvo se iz FAISS indeksa preuzima prvih k najslabijih čankova.
2. Dovućeni kontekst mora da upadne u limit modela. Kontekst se ne može slati neograničeno, već se računa raspoloživi budžet. Budžet se dobija tako što se od ukupnog limita oduzme broj tokena koji zauzima sam šablon prompta sa praznim kontekstom (fiksni, nepromenljivi deo) i mala sigurnosna rezerva SAFETY = 8 tokena, koja štiti od odstupanja

u tokenizaciji. Ostatak je broj tokena na raspolaganju za sadržaj dokumenata.

3. Punjenje konteksta. Pronađeni delovi radova se dodaju u kontekst jedan po jedan, i to tako što se dodaje naslov rada i tekst dovučenog komada teksta.
4. Podešavanja generisanja. Pri samom generisanju ulaz se dodatno osigurava skraćivanjem na maksimalan dozvoljen broj tokena. Koristi se *beam search* sa četiri grane (umesto da u svakom koraku bira najverovatniju reč, model paralelno prati četiri kandidat-niza i bira onaj sa najvećom ukupnom verovatnoćom). Ponavljanje istog niza od tri tokena je zabranjeno čime se sprečava da model upadne u petlju. Dužina izlaza ograničena je za dva zadatka, kraćim za klasifikaciju (gde se očekuje samo True ili False) i dužim za objašnjenje. Za objašnjenje se postavlja i minimalna dužina, da odgovor ne bude prekratak.

Pomenuti fiksni šablon je dat u nastavku:

```
STATUS_TEMPLATE = ""Is this claim true or false? Answer only True or False.
```

```
Evidence:
{context}
Claim: {claim}
Answer: ""
```

```
EXPLAIN_TEMPLATE = ""The claim is {verdict}. Based on the evidence, write one sentence explaining why. Start with "This claim is {verdict} because".
```

```
Evidence:
{context}
Claim: {claim}
Sentence: ""
```

Prva varijanta je čist FLAN-T5-XL, bez retrieval komponente. Modelu se prosleđuje prompt šablon, ali bez dela sa dokazima, tako da je jedina razlika prisustvo odnosno odsustvo konteksta, a ne formulacija zadatka. Odgovor se oslanja na znanje stečeno tokom predtreniranja, pa ova varijanta služi kao referentna vrednost koja pokazuje koliko model zna sam po sebi, bez pristupa spoljnim dokumentima.

Za drugu varijantu prikazan je sistem sa retrieverom (konkretno fine-tunovanim all-MiniLM-L6-v2). Poređenjem sa čistim modelom vidi se doprinos same *retrieval* komponente, odnosno koliko ubacivanje relevantnih dokumenata menja kvalitet i pouzdanost odgovora u odnosu na model prepušten sopstvenom znanju. Suštinski, baseline i fine tunovani model bi dali odgovor iste strukture, s tim da bi fine-tunovani model mogao bolje da pronađe relevantne dokumente (sudeći po test metrikama), pa je ovde prikazan slučaj njegovog korišćenja.

Prosleđena je ista tvrdnja za oba slučaja, preuzeta iz test upita:

Angiotensin converting enzyme inhibitors are associated with increased risk for functional renal insufficiency.

Scifact sadrži labelu za svaku od tvrdji i provereno je da je ova tvrdnja istinita (labela *True*).

Odgovor samog FLAN T5 je sledeći:

Status: False

Objašnjenje: This claim is false because Angiotensin converting enzyme inhibitors are not associated with increased risk for functional renal insufficiency.

Odgovor FLAN T5 sa ritriverom je:

This claim is true because angiotensin converting enzyme inhibitors are associated with increased risk for functional renal insufficiency.

Sources:

[1] Renal considerations in angiotensin converting enzyme inhibitor therapy: a statement for healthcare professionals from the Council on the Kidney in Cardiovascular Disease and the Council for High Blood Pressure Research of the American Heart Association.

[2] Differences in the Clinical Effects of Angiotensin-Converting Enzyme Inhibitors and Angiotensin Receptor Blockers: A Critical Review of the Evidence

Status: True

Kod čisto generativnog modela bez izvora, tvrdi se tačnost ili netačnost bez stvarnog osnova, što je i osnovna motivacija za uvođenje retrievala. Kod tvrdnji koje su modelu poznate iz predtreniranja kontekst ne mora praviti razliku, ali kod onih van njegovog znanja postaje presudan.

Treba naglasiti da prikazani odgovori, predstavljaju kvalitativnu ilustraciju na pojedinačnom primeru, a ne kvantitativno poređenje. Oni služe da prikažu razliku u ponašanju sistema, dok se pouzdano merenje doprinosa retrievala oslanja na retrieval metrike iz ranijeg dela rada.

Sam kvalitet generisanih odgovora, odnosno krajnjeg izlaza LLM-a, nije kvantitativno evaluiran, već je prikazan isključivo kroz pojedinačne primere.

Postoji više načina na koje bi se odgovori generatora mogli validirati u budućem radu. Mogla bi se primeniti evaluacija u kojoj drugi, jači jezički model ocenjuje kvalitet odgovora (tzv. LLM-as-a-judge), kao i ljudska procena na uzorku primera, koja ostaje najpouzdaniji, mada najskuplji, vid validacije.

X. LITERATURA

1. Thakur, N., Reimers, N., Rücklé, A., Srivastava, A., & Gurevych, I. (2021). *BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models*. <https://arxiv.org/abs/2104.08663>
2. BeIR/scifact dataset, Hugging Face: <https://huggingface.co/datasets/BeIR/scifact>
3. Vektor embedding model, all-MiniLM-L6-v2: [sentence-transformers/all-MiniLM-L6-v2](https://sentence-transformers.all-MiniLM-L6-v2) : Hugging Face
4. O *loss* funkciji *MultipleNegativesRankingLoss*: https://sbert.net/docs/package_reference/sentence_transformer/losses.html#multiplenegativesrankingloss